

Microservicio Leads B2B

API REST sin frontend para guardar empresas y generar correos breves de outreach B2B usando Groq.

Stack

- Node.js con ES Modules
- Express
- PostgreSQL
- pg
- Groq SDK
- Docker Compose
- SQL directo, sin ORM

Levantar con Docker Compose

Crear un archivo `.env` en la raíz:

```
GROQ_API_KEY=tu_api_key
GROQ_MODEL=llama-3.1-8b-instant
```

Luego ejecutar:

```
docker compose up --build
```

La API queda disponible en <http://localhost:3000>.

Variables de entorno

```
PORT=3000
DATABASE_URL=postgres://postgres@localhost:5432/tgp_leads
GROQ_API_KEY=your_groq_api_key_here
GROQ_MODEL=llama-3.1-8b-instant
```

En Docker Compose la API usa `DATABASE_URL=postgres://postgres@db:5432/tgp_leads`.

Nota: `POSTGRES_HOST_AUTH_METHOD=trust` se usa solo para desarrollo local. En producción se debe configurar usuario, contraseña y manejo seguro de credenciales.

Endpoints

GET /health

```
{  
  "status": "ok"  
}
```

POST /leads

Request:

```
{  
  "nombre_empresa": "Acme",  
  "dominio": "acme.com",  
  "cargo_contacto": "Gerente Comercial"  
}
```

Respuesta exitosa 201:

```
{  
  "data": {  
    "company": {  
      "id": 1,  
      "nombre_empresa": "Acme",  
      "dominio": "acme.com",  
      "cargo_contacto": "Gerente Comercial",  
      "status": "processed",  
      "created_at": "2026-06-15T00:00:00.000Z"  
    },  
    "outreach": {  
      "id": 1,  
      "correo_generado": "Hola...",  
      "llm_provider": "groq",  
      "llm_model": "llama-3.1-8b-instant",  
      "status": "generated",  
      "attempts": 1,  
      "created_at": "2026-06-15T00:00:00.000Z"  
    }  
  }  
}
```

GET /leads

Devuelve leads con correo generado, ordenados desde el más reciente.

Ejemplo de respuesta:

```
{
  "data": [
    {
      "company_id": 1,
      "nombre_empresa": "Acme",
      "dominio": "acme.com",
      "cargo_contacto": "Gerente Comercial",
      "lead_status": "processed",
      "outreach_id": 1,
      "correo_generado": "Hola, vi que Acme trabaja en acme.com...",
      "llm_provider": "groq",
      "llm_model": "llama-3.1-8b-instant",
      "status": "generated",
      "attempts": 1,
      "created_at": "2026-06-15T00:00:00.000Z",
      "updated_at": "2026-06-15T00:00:00.000Z"
    }
  ]
}
```

Decisiones de diseño

- La API separa responsabilidades entre rutas, controller, service y db.
- Las consultas SQL usan parámetros `$1`, `$2`, `$3`; no se concatenan valores de usuario.
- La empresa se guarda antes de llamar al LLM. Si Groq falla, el error queda asociado a la empresa.
- El estado del lead se guarda en `companies.status`: `pending` al recibirlo, `processed` si se genera el correo y `error` si falla el LLM.
- `GET /leads` lista solo leads con correo generado, no intentos fallidos.
- Los retries del LLM son para resiliencia ante fallos temporales, no para resolver escalabilidad masiva.

Manejo de errores

- Los controllers usan `try/catch` y delegan en el middleware centralizado.
- Los errores no exponen stack trace al cliente.
- Los errores se loguean en consola con método y ruta.
- Si Groq falla tras 3 intentos, se guarda en `lead_errors` con `stage = "LLM_GENERATION"` y `attempts = 3`, y se responde `502`.

Mitigación de prompt injection

- Validación de tipos, campos obligatorios y largos máximos.
- Dominio con formato razonable.
- Sanitización básica de strings.
- Rechazo de patrones sospechosos como `ignora instrucciones`, `system prompt` o `ignore previous instructions`.
- Prompt con datos delimitados en `<lead_data>`.

- El system prompt indica que los datos del lead son externos y nunca deben tratarse como instrucciones.
- El prompt evita placeholders como [Tu nombre], no inventa experiencia previa, cifras ni clientes similares, y personaliza solo con nombre_empresa, dominio y cargo_contacto.

Qué mejoraría con más tiempo

- Tests automatizados con base de datos temporal.
- Timeouts explícitos para llamadas a Groq.
- Rate limiting por IP o API key.
- Cola de trabajos para desacoplar la generación del correo.
- Observabilidad estructurada con métricas y trazas.
- Conectar con una base interna que tenga contexto de proyectos para entregar más información útil al modelo, manteniendo datos actualizados sobre las compañías que maneja la empresa.

Preguntas técnicas

1. Escala: 500 requests en 1 segundo

Si el endpoint recibiera 500 requests en 1 segundo, probablemente se saturaría primero la dependencia con el LLM por latencia y rate limits; después podría saturarse el pool de PostgreSQL si cada request escribe inmediatamente. Para escalarlo, separaría la recepción del lead de la generación del correo: `POST /leads` validaría, guardaría el lead en estado `pending` y respondería `202 Accepted`; luego una cola procesaría los leads con workers, límites de concurrencia, rate limiting, timeouts, retries con backoff y caché para evitar regenerar correos repetidos.

2. Prompt injection

El riesgo principal es que campos externos como `nombre_empresa` o `cargo_contacto` incluyan instrucciones maliciosas y el modelo las interprete como parte del prompt. Para mitigarlo se validan tipos, largos máximos y formato, se detectan patrones sospechosos básicos, se delimitan los datos en `<lead_data>` como información externa no confiable y se usa un system prompt defensivo; no dependería solo de una blacklist porque siempre pueden aparecer nuevas formas de escribir una instrucción maliciosa.

3. Resiliencia y costo del LLM

En producción manejaría las llamadas al LLM con timeouts, máximo 3 reintentos y backoff para no insistir inmediatamente ante errores temporales o rate limits. Para controlar costos, limitaría el tamaño del input, los tokens de salida y la temperatura; además registraría tokens usados, costo estimado, modelo utilizado y cantidad de intentos, evaluando caché o modelos más baratos para tareas simples.

4. Observabilidad y calidad

Registraría por cada llamada un `request_id`, `company_id`, versión del prompt, modelo usado, cantidad de intentos, latencia, estado final, tokens de entrada y salida, y si la respuesta pasó validaciones mínimas de calidad. Además, mantendría métricas agregadas de las últimas llamadas, por ejemplo tiempo promedio de respuesta y tokens promedio de las últimas 10 generaciones; si esos valores suben sobre un

umbral definido, levantaría una alerta. Para calidad, revisaría manualmente una muestra aleatoria de respuestas y marcaría respuestas sospechosas por largo, tono o falta de personalización.

5. Producción en AWS

Para desplegarlo en AWS usaría ECS Fargate para correr la API y los workers sin administrar servidores, RDS PostgreSQL para la base de datos, SQS para desacoplar la generación del correo, CloudWatch para logs y métricas, Secrets Manager para credenciales y un Load Balancer para exponer la API.